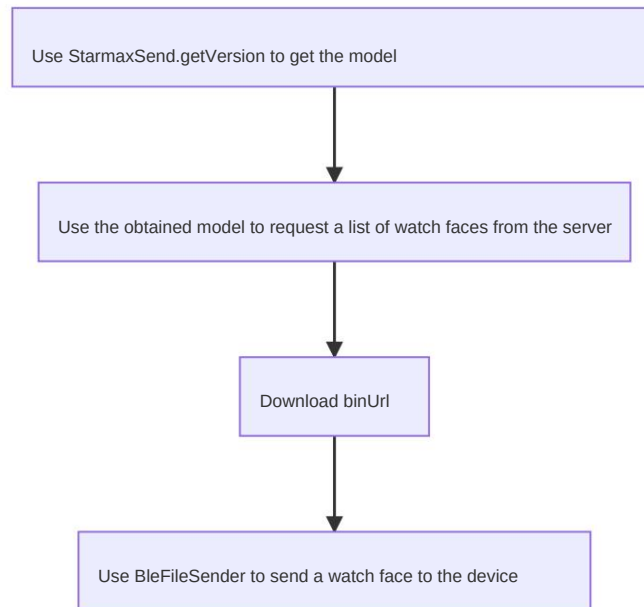


Watchface download and OTA instructions

1. Interface Usage

1.1 Flowchart of dial usage:



1.2 Network download example

```
DialRepository.getLists("X01M11T001", "v1.7.6", onSuccess = {
    result, response ->
    println(result)
}, onError = {
    it?.printStackTrace()
})
```

1.3 Sending watch face example

```
fun sendDialLocal(context: Context) { //Need to request
    //MTU first
    changeMtu {
        try { //
            //Convert the downloaded dial to FileInputStream
            val bin = context.contentResolver.openInputStream(binUri!!)
            as FileInputStream?
            BleFileSender.initFile(
                bin,
                object : BleFileSenderListener() { override fun
                    onSuccess() { //Send successfully
                }
            }
        }
    }
}
```

```

        override fun onProgress(progress: Double) {
            //Get the sending progress 0--100
        }

        override fun onFailure() { //Send failed
        }

        override fun onStart() {
            val data = StarmaxSend()
                .sendDial(
                    30000,
                    BmpUtils.bmp24to16(255, 255, 255),
                    1
                ) //Send a request header with dial information to Bluetooth
            sendMsg(data)
        }

        override fun onSend() {
            if (BleFileSender.hasNext()) {
                //Send file fragment data to Bluetooth
                val data = StarmaxSend().sendFile()
                sendMsg(data)
            }
        }
    })

    //Fixed value 8
    BleFileSender.sliceBuffer = 8

    //Start sending
    BleFileSender.onStart()
} catch (e: FileNotFoundException) { e.printStackTrace()
}
}
}
}

```

2. Usage Class Description

2.1 Request a network watch face

DialRepository

2.1.1 Get all watch faces

```

fun DialRepository.getLists(model: String, onSuccess: (List<Dial>?, AbstractResponse<*>) ->
Unit, onError: (e: Exception?) -> Unit,
phaseResponse: Boolean = true)

```

Field Description:

Fields	property	illustrate	enumerate
model	String	Equipment batch number	like X01M01T001
onSuccess	(List?,AbstractResponse<*>) -> Unit	Return to Dial List	
onError	(e: Exception?) -> Unit	Is it a mobile phone number?	
phaseResponse	Boolean	Whether to parse in advance List, usually true	

2.1.2 Get the classification dial

```
fun DialRepository.getLabels(model: String, lang: String, onSuccess:  
(List<DialLabel>?,AbstractResponse<*>) -> Unit, onError: (e: Exception?) ->  
Unit, phaseResponse: Boolean = true)
```

Field Description:

Fields	property	Description Enumeration
model	String	equipment batch Number Such as X01M01T001
just	String	ÿÿ zh_CN,zh_HK,en,ru,fr,es,pt,de,ja,pl,it,ko,ar,nl,th,hi_IN,in,vi,hi
onSuccess	(List?,AbstractResponse<*>) -> Unit	return Dial List
onError	(e: Exception?) -> Unit	whether For Hand Machine number
phaseResponse	Boolean	whether in advance Analysis for List, generally for true

2.1.3 Watch face information

```
class Dial {  
    var title: String  
    var customId: Int  
    var labelId: Int  
    var picUrl: String  
    var binUrl: String  
}
```

Field Description:

Fields	property	illustrate	enumerate
title	String	Dial Name	
customId	Int	Dial ID	
labelId	Int	Category ID	
picUrl	String	Thumbnail URL	
binUrl	String	Watch face bin file	

2.1.4 Watch face classification information

```
class DialLabel(data: Client.DialLabel) {  
    where id: Int  
    var title: String  
    var dials: List<Dial>  
}
```

Field Description:

Fields	property	illustrate	enumerate
id	Int	CategoryId	
title	String	Category Name	
dials	List	Watch Face List	

2.2 Sending Files

2.2.1 Send file monitoring

```
abstract class BleFileSenderListener {  
    abstract fun onSuccess()  
  
    abstract fun onProgress(progress: Double)  
  
    abstract fun onFailure()  
  
    abstract fun onSend()  
  
    abstract fun onStart()  
  
}
```

Method Description:

Fields	illustrate
onSuccess()	Send success callback
onProgress(progress: Double)	Send progress callback
onFailure()	Sending failure callback
onSend()	Send callback, users need to process the sending file fragments here
onStart()	Start sending callback, where the user sends file header information to Bluetooth

2.2.2 Sending Files

```
object BleFileSender {  
    fun initFile(inputStream: InputStream?, bleFileSenderListener:  
BeFileSenderListener)  
  
    fun initFileWithBackground(binInputStream:  
InputStream?,width:Int,height:Int,imageInputStream: InputStream?,  
bleFileSenderListener: bleFileSenderListener)  
  
    fun onStart()  
  
    fun sendFile(): ByteArray  
  
    fun hasNext(): Boolean  
  
}
```

Method Description:

Fields	illustrate
initFile	Initialize file stream
initFileWithBackground initializes the file stream and image stream, which is mainly used for customizing the dial. Here, width and Height refers to the screen size, which needs to be obtained using StarmaxSend.getVersion	
onStart	Start sending
sendFile	File fragment data
hasNext	Monitors whether the file has been sent. If true, it means there is another packet of data.

2.3 Send file header information

This type of interface is called in BleFileSender.onStart. The device uses this information to distinguish dials, UI, firmware, etc.

2.3.1 Sending a dial

```
StarmaxSend().sendDial(dialId: Int, color: Int, align: Int): ByteArray
```

Field Description:

Fields	property	illustrate	enumerate
dialId	Int	Dial Id	
color	Int	color	
align	Int	Text Position	

2.3.2 Send UI

```
StarmaxSend().sendUi(offset: Int, version: String): ByteArray
```

Field Description:

Fields	property	illustrate
offset	Int	Offset

Fields	property	illustrate
version	String	Version Number

2.3.3 Send firmware (applicable to S5-4)

StarmaxSend().sendFirmware(): ByteArray

2.4 Sending file fragments

This type of interface is called in BleFileSender.onSend

StarmaxSend().sendFile(): ByteArray